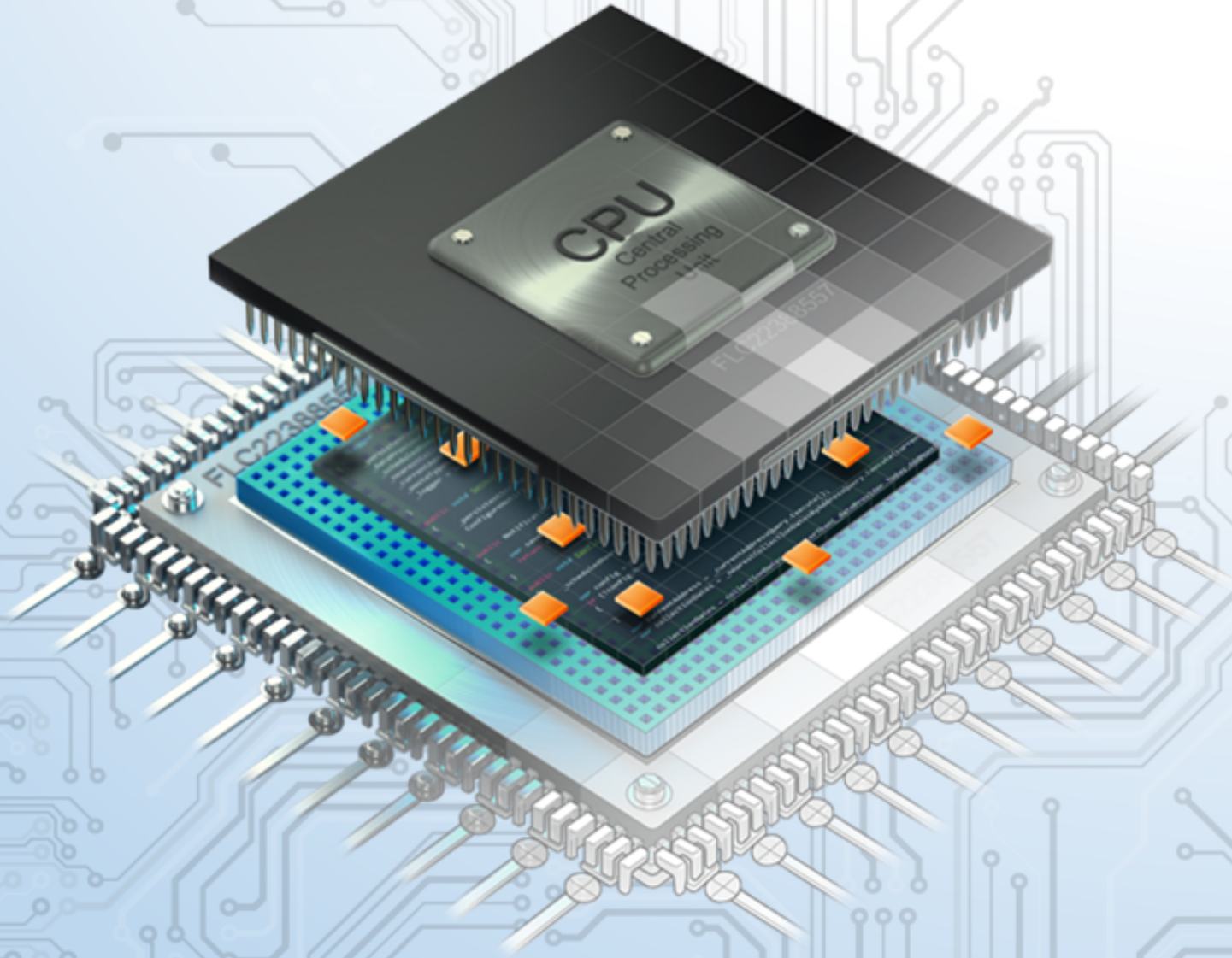
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
COMPUTER ENGINEERING

Microcontroller



Dr. Le Trong Nhan

Mục lục

Chapter 1. Buttons/Switches **5**

1 Objectives 6

2 Introduction 6

3 Basic techniques for reading from port pins 8

3.1 The need for pull-up resistors 8

3.2 Dealing with switch bounces 8

4 Reading switch input (basic code) using STM32 13

4.1 Input Output Processing Patterns 13

4.2 Setting up 14

4.2.1 Create a project 14

4.2.2 Create a file C source file and header file for input reading 14

4.3 Code For Read Port Pin and Debouncing 16

4.3.1 The code in the input_reading.c file 16

4.3.2 The code in the input_reading.h file 17

4.3.3 The code in the timer.c file 17

4.4 Button State Processing 18

4.4.1 Finite State Machine 18

4.4.2 The code for the FSM in the input_processing.c file 19

4.4.3 The code in the input_processing.h 19

4.4.4 The code in the main.c file 20

5 Exercises and Report 21

5.1 Specifications 21

5.2 Exercise 1: Sketch an FSM 22

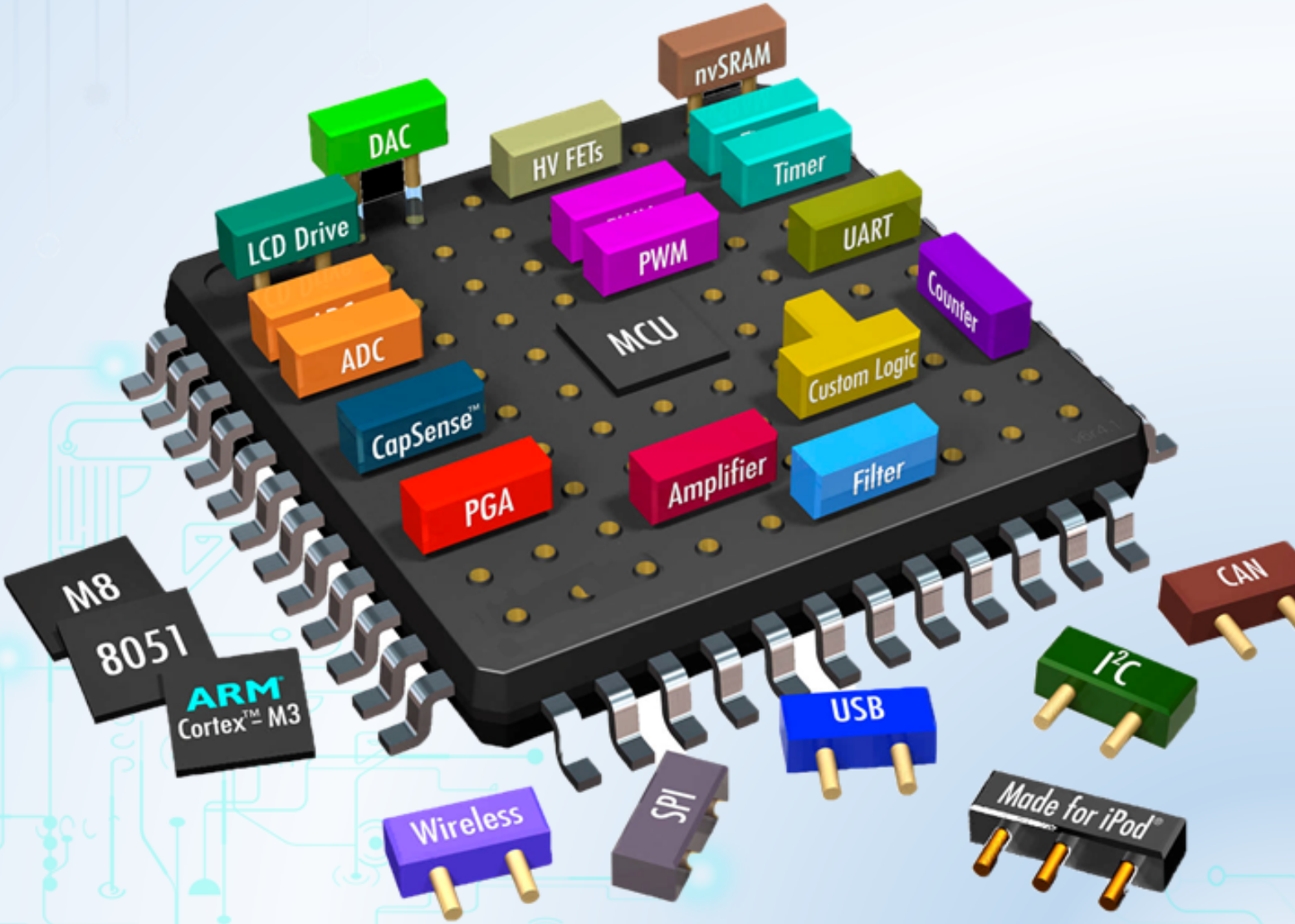
5.3 Exercise 2: Proteus Schematic 23

5.4 Exercise 3: Create STM32 Project 23

5.5 Exercise 4: Modify Timer Parameters 23
5.6 Exercise 5: Adding code for button debouncing 25
5.7 Exercise 6: Adding code for displaying modes 26
5.8 Exercise 7: Adding code for increasing time duration value for the red LEDs 27
5.9 Exercise 8: Adding code for increasing time duration value for the amber LEDs 28
5.9.1 The code in the main.c file 28
5.10 Exercise 9: Adding code for increasing time duration value for the green LEDs 28
5.11 Exercise 10: To finish the project 29

CHƯƠNG 1

Buttons/Switches



1 Objectives

In this lab, you will

- Learn how to add new C source files and C header files in an STM32 project,
- Learn how to read digital inputs and display values to LEDs using a timer interrupt of a microcontroller (MCU).
- Learn how to debounce when reading a button.
- Learn how to create an FSM and implement an FSM in an MCU.

2 Introduction

Embedded systems usually use buttons (or keys, or switches, or any form of mechanical contacts) as part of their user interface. This general rule applies from the most basic remote-control system for opening a garage door, right up to the most sophisticated aircraft autopilot system. Whatever the system you create, you need to be able to create a reliable button interface.

A button is generally hooked up to an MCU so as to generate a certain logic level when pushed or closed or “active" and the opposite logic level when unpushed or open or “inactive." The active logic level can be either ‘0’ or ‘1’, but for reasons both historical and electrical, an active level of ’0’ is more common.

We can use a button if we want to perform operations such as:

- Drive a motor while a switch is pressed.
- Switch on a light while a switch is pressed.
- Activate a pump while a switch is pressed.

These operations could be implemented using an electrical button without using an MCU; however, use of an MCU may well be appropriate if we require more complex behaviours. For example:

- Drive a motor while a switch is pressed.

  Condition: If the safety guard is not in place, don’t turn the motor. Instead sound a buzzer for 2 seconds.

- Switch on a light while a switch is pressed.

  Condition: To save power, ignore requests to turn on the light during daylight hours.

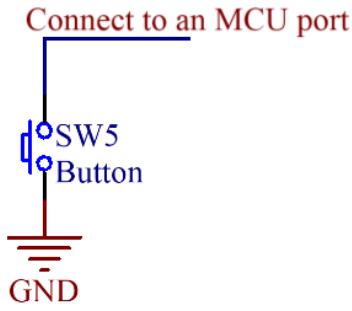
- Activate a pump while a switch is pressed

  Condition: If the main water reservoir is below 300 litres, do not start the main pump: instead, start the reserve pump and draw the water from the emergency tank.

In this lab, we consider how you read inputs from mechanical buttons in your embedded application using an MCU.

3 Basic techniques for reading from port pins

3.1 The need for pull-up resistors



Hình 1.1: *Connecting a button to an MCU*

Figure 1.1 shows a way to connect a button to an MCU. This hardware operates as follows:

- When the switch is open, it has no impact on the port pin. An internal resistor on the port “pulls up" the pin to the supply voltage of the MCU (typically 3.3V for STM32F103). If we read the pin, we will see the value ‘1’.
- When the switch is closed (pressed), the pin voltage will be 0V. If we read the pin, we will see the value ‘0’.

However, if the MCU does not have a pull-up resistor inside, when the button is pressed, the read value will be ‘0’, but even we release the button, the read value is still ‘0’ as shown in Figure 1.2.

So a reliable way to connect a button/switch to an MCU is that we explicitly use an external pull-up resistor as shown in Figure 1.3.

3.2 Dealing with switch bounces

In practice, all mechanical switch contacts bounce (that is, turn on and off, repeatedly, for short period of time) after the switch is closed or opened as shown in Figure 1.4.

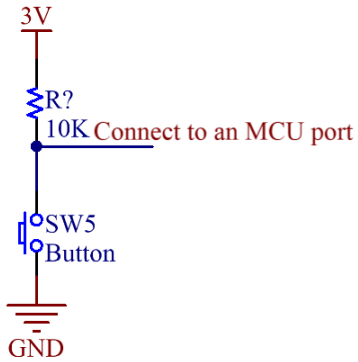
Every system that uses any kind of mechanical switch must deal with the issue of debouncing. The key task is to make sure that one mechanical switch or button action is only read as one action by the MCU, even though the MCU will typically be fast enough to detect the unwanted switch bounces and treat them as separate events. Bouncing can be eliminated by special ICs or by RC circuitry, but in most cases debouncing is done in software because software is “free".

As far as the MCU concerns, each “bounce" is equivalent to one press and release of an “ideal” switch. Without appropriate software design, this can give several problems:

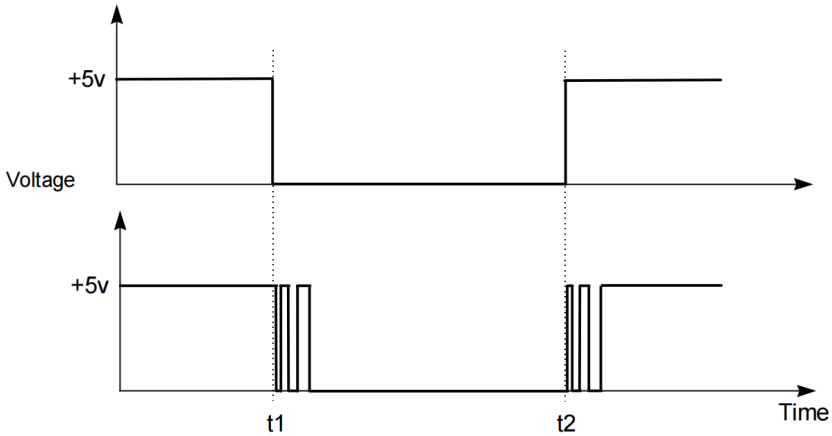
With pull-ups:

Without pull-ups:

Hình 1.2: The need of pull up resistors



Hình 1.3: A reliable way to connect a button to an MCU



Hình 1.4: Switch bounces

- Rather than reading 'A' from a keypad, we may read 'AAAAA'
- Counting the number of times that a switch is pressed becomes extremely difficult
- If a switch is depressed once, and then released some time later, the 'bounce' may make it appear as if the switch has been pressed again (at the time of release).

The key to debouncing is to establish a minimum criterion for a valid button push, one that can be implemented in software. This criterion must involve differences in time - two button presses in 20ms must be treated as one button event, while two button presses in 2 seconds must be treated as two button events. So what are the relevant times we need to consider? They are these:

- Bounce time: most buttons seem to stop bouncing within 10ms
- Button press time: the shortest time a user can press and release a button seems to be between 50 and 100ms
- Response time: a user notices if the system response is 100ms after the button press, but not if it is 50ms after

Combining all of these times, we can set a few goals

- Ignore all bouncing within 10ms
- Provide a response within 50ms of detecting a button push (or release)
- Be able to detect a 50ms push and a 50ms release

The simplest debouncing method is to examine the keys (or buttons or switches) every N milliseconds, where N > 10ms (our specified button bounce upper limit) and N <= 50ms (our specified response time). We then have three possible outcomes every time we read a button:

- We read the button in the solid '0' state
- We read the button in the solid '1' state
- We read the button while it is bouncing (so we will get either a '0' or a '1')

Outcomes 1 and 2 pose no problems, as they are what we would always like to happen. Outcome 3 also poses no problem because during a bounce either state is acceptable. If we have just pressed an active-low button and we read a '1' as it bounces, the next time through we are guaranteed to read a '0' (remember, the next time through all bouncing will have ceased), so we will just detect the button push a bit later. Otherwise, if we read a '0' as the button bounces, it will still be '0' the next time after all bouncing has stopped, so we are just detecting the button push a bit earlier. The same applies to releasing a button. Reading a single bounce (with all bouncing over by the time of the next read) will never give us an invalid button state. It's only reading multiple bounces (multiple reads while bouncing is

occurring) that can give invalid button states such as repeated push signals from one physical push.

So if we guarantee that all bouncing is done by the time we next read the button, we're good. Well, almost good, if we're lucky...

MCUs often live among high-energy beasts, and often control the beasts. High energy devices make electrical noise, sometimes great amounts of electrical noise. This noise can, at the worst possible moment, get into your delicate button-and-high-value-pullup circuit and act like a real button push. Oops, missile launched, sorry!

If the noise is too intense we cannot filter it out using only software, but will need hardware of some sort (or even a redesign). But if the noise is only occasional, we can filter it out in software without too much bother. The trick is that instead of regarding a single button 'make' or 'break' as valid, we insist on N contiguous makes or breaks to mark a valid button event. N will be a factor of your button scanning rate and the amount of filtering you want to add. Bigger N gives more filtering. The simplest filter (but still a big improvement over no filtering) is just an N of 2, which means compare the current button state with the last button state, and only if both are the same is the output valid.

Note that now we have not two but three button states: active (or pressed), inactive (or released), and indeterminate or invalid (in the middle of filtering, not yet filtered). In most cases we can treat the invalid state the same as the inactive state, since we care in most cases only about when we go active (from whatever state) and when we cease being active (to inactive or invalid). With that simplification we can look at simple N = 2 filtering reading a button wired to STM32 MCU:

```
void button_reading(void){
    static unsigned char last_button;
    unsigned char raw_button;
    unsigned char filtered_button;
    last_button = raw_button;
    raw_button = HAL_GPIO_ReadPin(BUTTON_1_GPIO_Port,
   BUTTON_1_Pin);
    if(last_button == raw_button){
        filtered_button = raw_button;
    }
}
```

Program 1.1: Read port pin and deboucing

The function button_reading() must be called no more often than our debounce time (10ms).

To expand to greater filtering (larger N), keep in mind that the filtering technique essentially involves reading the current button state and then either counting or reseting the counter. We count if the current button state is the same as the last button state, and if our count reaches N we then report a valid new button state. We reset the counter if the current button state is different than the last button state, and we then save the current button state as the new button state to compare against the next time. Also note that the larger our value of N the more often our filtering routine must be called, so that we get a filtered response within our

specified 50ms deadline. So for example with an N of 8 we should be calling our filtering routine every 2 - 5ms, giving a response time of 16 - 40ms (>10ms and <50ms).

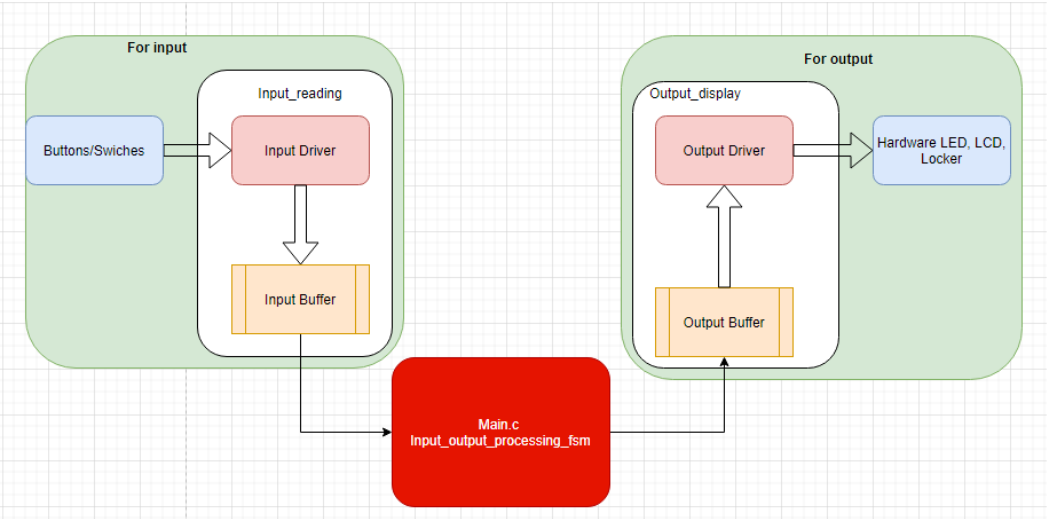
4 Reading switch input (basic code) using STM32

To demonstrate the use of buttons/switches in STM32, we use an example which requires to write a program that

- Has a timer which has an interrupt in every 10 milliseconds.
- Reads values of button PB0 every 10 milliseconds.
- Increases the value of LEDs connected to PORTA by one unit when the button PB0 is pressed.
- Increases the value of PORTA automatically in every 0.5 second, if the button PB0 is pressed in more than 1 second.

4.1 Input Output Processing Patterns

For both input and output processing, we have a similar pattern to work with. Normally, we have a module named driver which works directly to the hardware. We also have a buffer to store temporarily values. In the case of input processing, the driver will store the value of the hardware status to the buffer for further processing. In the case of output processing, the driver uses the buffer data to output to the hardware.



Hình 1.5: Input Output Processing Patterns

Figure 1.5 shows that we should have an *input_reading* module to processing the buttons, then store the processed data to the buffer. Then a module of *input_output_processin* will process the input data, and update the output buffer. The output driver gets the value from the output buffer to transfer to the hardware.

4.2 Setting up

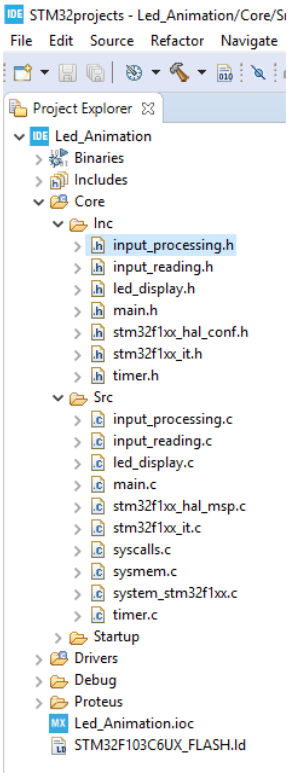
4.2.1 Create a project

Please follow the instruction in Labs 1 and 2 to create a project that includes:

- PB0 as an input port pin,
- PA0-PA7 as output port pins, and
- Timer 2 10ms interrupt

4.2.2 Create a file C source file and header file for input reading

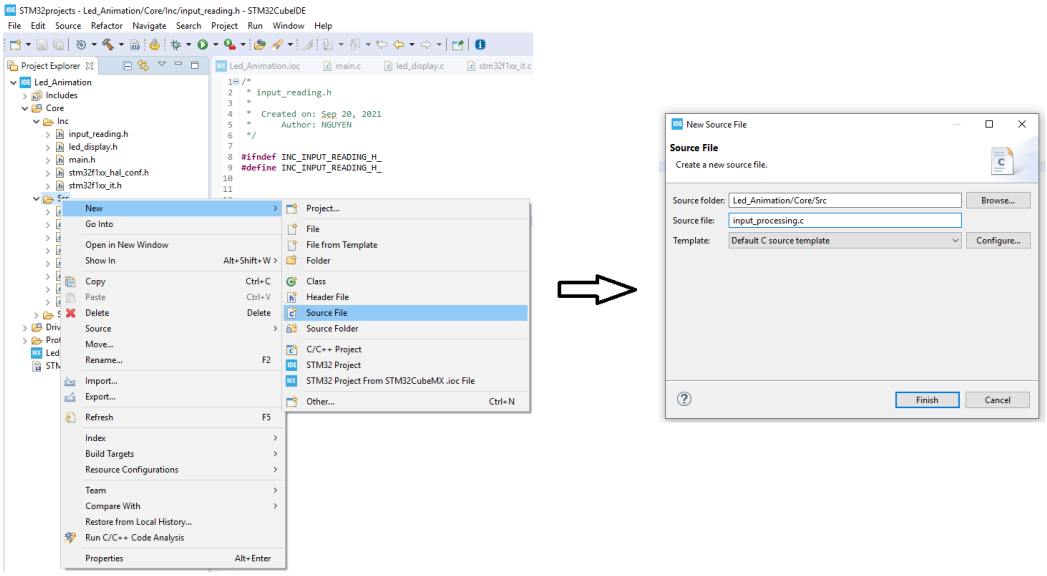
We are expected to have files for button processing and led display as shown in Figure 1.6.



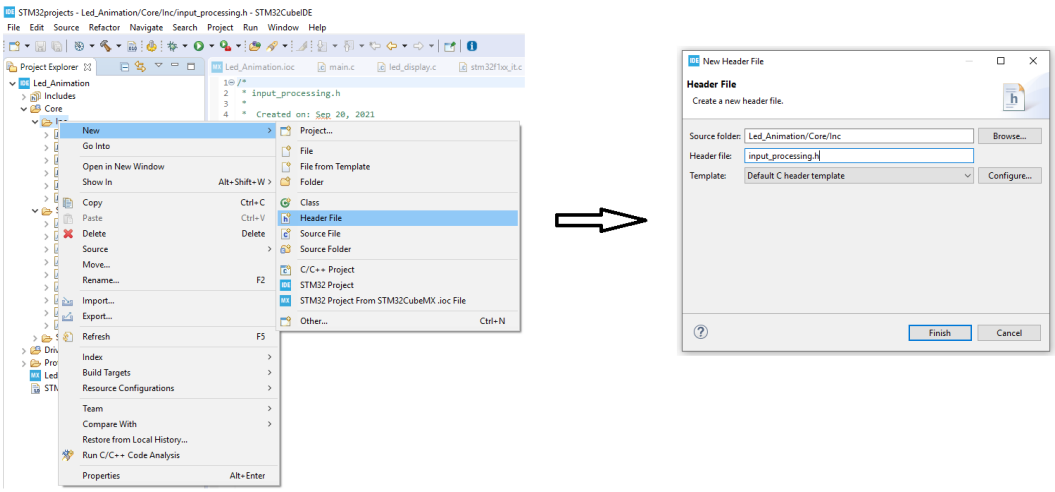
Hình 1.6: File Organization

Steps 1 (Figure 1.7): Right click to the folder **Src**, select **New**, then select **Source File**. There will be a pop-up. Please type the file name, then click **Finish**.

Step 2 (Figure 1.8): Do the same for the C header file in the folder **Inc**.



Hình 1.7: Step 1: Create a C source file for input reading



Hình 1.8: Step 2: Create a C header file for input processing

4.3 Code For Read Port Pin and Debouncing

4.3.1 The code in the input_reading.c file

```c
#include "main.h"
//we aim to work with more than one buttons
#define NO_OF_BUTTONS                    1
//timer interrupt duration is 10ms, so to pass 1 second,
//we need to jump to the interrupt service routine 100 time
#define DURATION_FOR_AUTO_INCREASING     100
#define BUTTON_IS_PRESSED                  GPIO_PIN_RESET
#define BUTTON_IS_RELEASED                 GPIO_PIN_SET
//the buffer that the final result is stored after
//debouncing
static GPIO_PinState buttonBuffer[NO_OF_BUTTONS];
//we define two buffers for debouncing
static GPIO_PinState debounceButtonBuffer1[NO_OF_BUTTONS];
static GPIO_PinState debounceButtonBuffer2[NO_OF_BUTTONS];
//we define a flag for a button pressed more than 1 second.
static uint8_t flagForButtonPress1s[NO_OF_BUTTONS];
//we define counter for automatically increasing the value
//after the button is pressed more than 1 second.
static uint16_t counterForButtonPress1s[NO_OF_BUTTONS];
void button_reading(void){
  for(char i = 0; i < NO_OF_BUTTONS; i ++){
    debounceButtonBuffer2[i] =debounceButtonBuffer1[i];
    debounceButtonBuffer1[i] = HAL_GPIO_ReadPin(BUTTON_1_GPIO_Port, BUTTON_1_Pin);
    if(debounceButtonBuffer1[i] == debounceButtonBuffer2[i])
      buttonBuffer[i] = debounceButtonBuffer1[i];
      if(buttonBuffer[i] == BUTTON_IS_PRESSED){
      //if a button is pressed, we start counting
        if(counterForButtonPress1s[i] < DURATION_FOR_AUTO_INCREASING){
          counterForButtonPress1s[i]++;
        } else {
        //the flag is turned on when 1 second has passed
        //since the button is pressed.
          flagForButtonPress1s[i] = 1;
          //todo
        }
      } else {
        counterForButtonPress1s[i] = 0;
        flagForButtonPress1s[i] = 0;
      }
  }
}
```

Program 1.2: Define constants buffers and button_reading function

```c
unsigned char is_button_pressed(uint8_t index){
  if(index >= NO_OF_BUTTONS) return 0;
  return (buttonBuffer[index] == BUTTON_IS_PRESSED);
}
```

Program 1.3: Checking a button is pressed or not

```c
unsigned char is_button_pressed_1s(unsigned char index){
  if(index >= NO_OF_BUTTONS) return 0xff;
  return (flagForButtonPress1s[index] == 1);
}
```

Program 1.4: Checking a button is pressed more than a second or not

4.3.2 The code in the input_reading.h file

```c
#ifndef INC_INPUT_READING_H_
#define INC_INPUT_READING_H_
void button_reading(void);
unsigned char is_button_pressed(unsigned char index);
unsigned char is_button_pressed_1s(unsigned char index);
#endif /* INC_INPUT_READING_H_ */
```

Program 1.5: Prototype in input_reading.h file

4.3.3 The code in the timer.c file

```c
#include "main.h"
#include "input_reading.h"

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
  if(htim->Instance == TIM2){
    button_reading();
  }
}
```

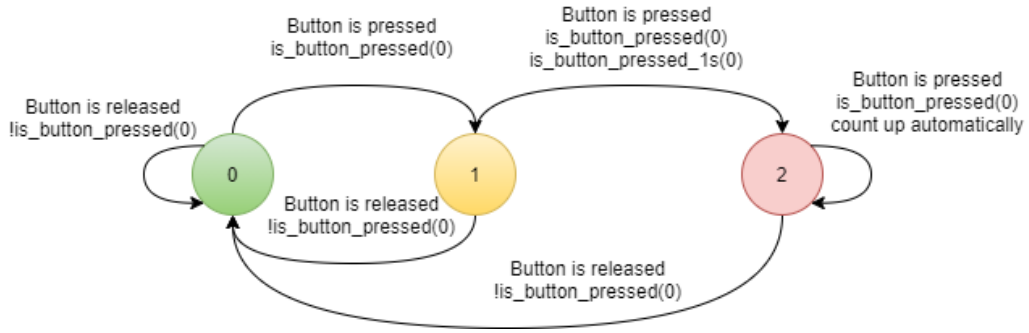
Program 1.6: Timer interrupt callback function

4.4 Button State Processing

4.4.1 Finite State Machine

To solve the example problem, we define 3 states as follows:

- State 0: The button is released or the button is in the initial state.
- State 1: When the button is pressed, the FSM will change to State 1 that is increasing the values of PORTA by one value. If the button is released, the FSM goes back to State 0.
- State 2: while the FSM is in State 1, the button is kept pressing more than 1 second, the state of FSM will change from 1 to 2. In this state, if the button is kept pressing, the value of PORTA will be increased automatically in every 500ms. If the button is released, the FSM goes back to State 0.



Hình 1.9: An FSM for processing a button

4.4.2 The code for the FSM in the input_processing.c file

Please note that *fsm_for_input_processing* function should be called inside the super loop of the main functin.

```c
#include "main.h"
#include "input_reading.h"

enum ButtonState{BUTTON_RELEASED, BUTTON_PRESSED,
   BUTTON_PRESSED_MORE_THAN_1_SECOND} ;
enum ButtonState buttonState = BUTTON_RELEASED;
void fsm_for_input_processing(void){
  switch(buttonState){
  case BUTTON_RELEASED:
    if(is_button_pressed(0)){
      buttonState = BUTTON_PRESSED;
      //INCREASE VALUE OF PORT A BY ONE UNIT
    }
    break;
  case BUTTON_PRESSED:
    if(!is_button_pressed(0)){
      buttonState = BUTTON_RELEASED;
    } else {
      if(is_button_pressed_1s(0)){
        buttonState = BUTTON_PRESSED_MORE_THAN_1_SECOND;
      }
    }
    break;
  case BUTTON_PRESSED_MORE_THAN_1_SECOND:
    if(!is_button_pressed(0)){
      buttonState = BUTTON_RELEASED;
    }
    //todo
    break;
  }
}
```

Program 1.7: The code in the input_processing.c file

4.4.3 The code in the input_processing.h

```c
#ifndef INC_INPUT_PROCESSING_H_
#define INC_INPUT_PROCESSING_H_

void fsm_for_input_processing(void);

#endif /* INC_INPUT_PROCESSING_H_ */
```

Program 1.8: Code in the input_processing.h file

4.4.4 The code in the main.c file

```c
#include "main.h"
#include "input_processing.h"
//don't modify this part
int main(void){
    HAL_Init();
    /* Configure the system clock */
    SystemClock_Config();
    /* Initialize all configured peripherals */
    MX_GPIO_Init();
    MX_TIM2_Init();
    while (1)
    {
        //you only need to add the fsm function here
        fsm_for_input_processing();
    }
}
```

Program 1.9: The code in the main.c file

5 Exercises and Report

5.1 Specifications

You are required to build an application of a traffic light in a cross road which includes some features as described below:

- The application has 12 LEDs including 4 red LEDs, 4 amber LEDs, 4 green LEDs.
- The application has 4 seven segment LEDs to display time with 2 for each road. The 2 seven segment LEDs will show time for each color LED corresponding to each road.
- The application has three buttons which are used

  - to select modes,

  - to modify the time for each color led on the fly, and

  - to set the chosen value.
- The application has at least 4 modes which is controlled by the first button. Mode 1 is a normal mode, while modes 2 3 4 are modification modes. You can press the first button to change the mode. Modes will change from 1 to 4 and back to 1 again.

  Mode 1 - Normal mode:

  - The traffic light application is running normally.

  Mode 2 - Modify time duration for the red LEDs: This mode allows you to change the time duration of the red LED in the main road. The expected behaviours of this mode include:

  - All single red LEDs are blinking in 2 Hz.

  - Use two seven-segment LEDs to display the value.

  - Use the other two seven-segment LEDs to display the mode.

  - The second button is used to increase the time duration value for the red LEDs.

  - The value of time duration is in a range of 1 - 99.

  - The third button is used to set the value.

  Mode 3 - Modify time duration for the amber LEDs: Similar for the red LEDs described above with the amber LEDs.

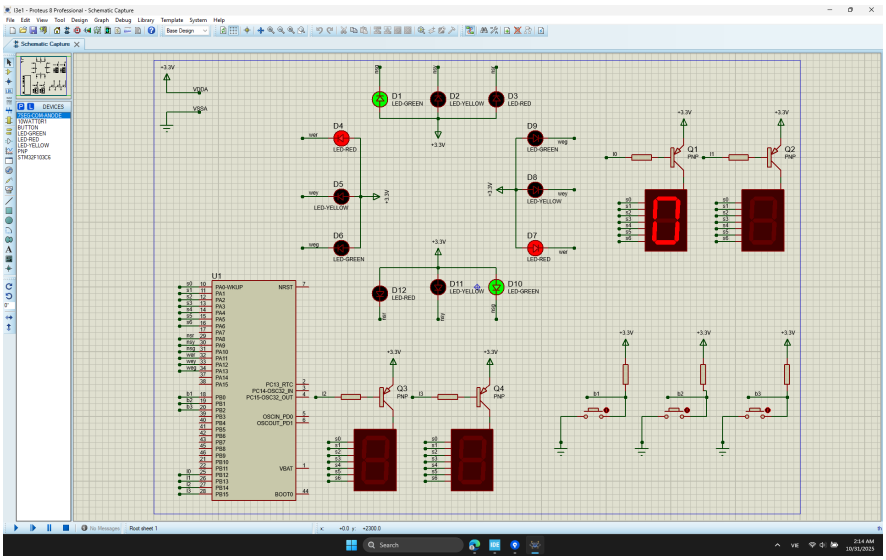
  Mode 4 - Modify time duration for the green LEDs: Similar for the red LEDs described above with the green LEDs.

5.2 Exercise 1: Sketch an FSM

Your task in this exercise is to sketch an FSM that describes your idea of how to solve the problem.

Please add your report here.

State	Description	Transition Condition	Action
State 0	Button is released (initial state)	Button pressed → State 1	No action
State 1	Button pressed (less than 1 second)	Button released → State 0 Button pressed > 1s → State 2	Increment PORTA by 1
State 2	Button pressed for more than 1 second	Button released → State 0	Auto-increment PORTA every 500ms



5.3 Exercise 2: Proteus Schematic

Your task in this exercise is to draw a Proteus schematic for the problem above.

Please add your report here.

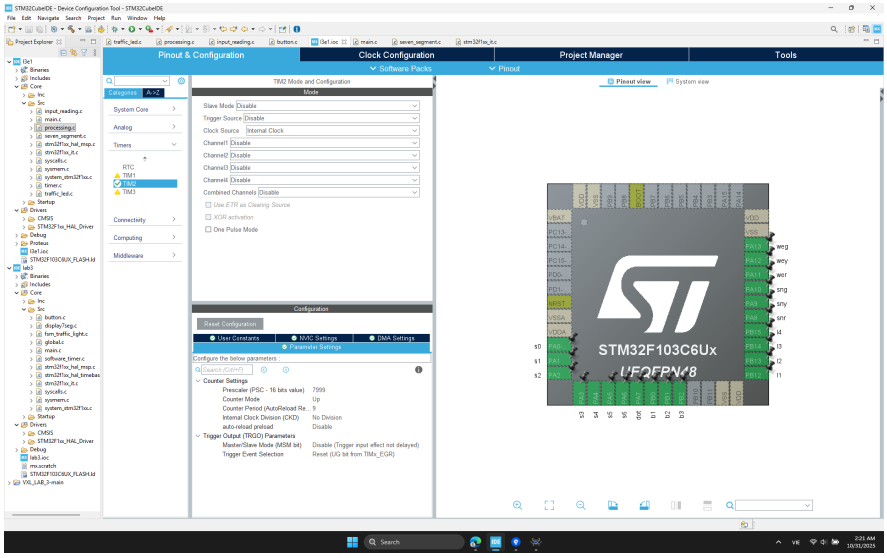
5.4 Exercise 3: Create STM32 Project

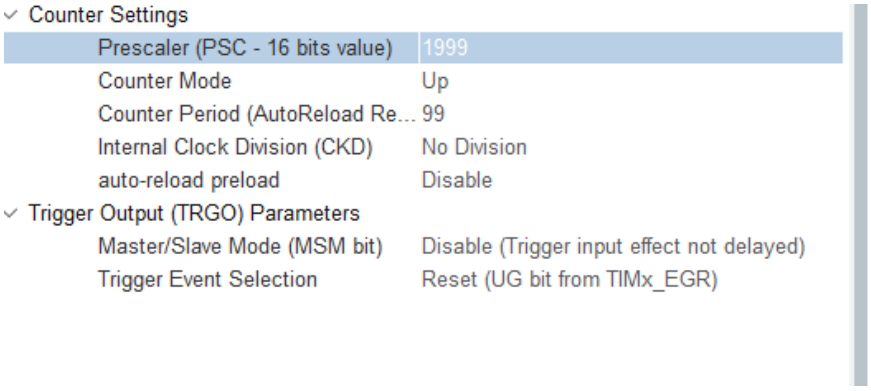
Your task in this exercise is to create a project that has pin corresponding to the Proteus schematic that you draw in previous section. You need to set up your timer interrupt is about 10ms.

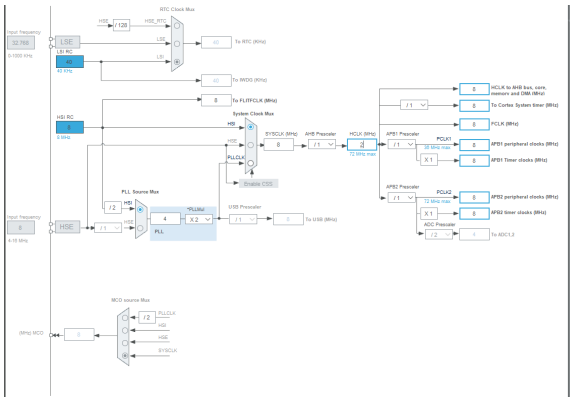
Please add your report here.

5.5 Exercise 4: Modify Timer Parameters

Your task in this exercise is to modify the timer settings so that when we want to change the time duration of the timer interrupt, we change it the least and it will not affect the overall system. For example, the current system we have implemented is that it can blink an LED in 2 Hz, with the timer interrupt duration is 10ms. However, when we want to change the timer interrupt duration to 1ms or 100ms, it will not affect the 2Hz blinking LED. Please add your report here.







Hình 1.10: Enter Caption

5.6 Exercise 5: Adding code for button debouncing

Following the example of button reading and debouncing in the previous section, your tasks in this exercise are:

- To add new files for input reading and output display,
- To add code for button debouncing,
- To add code for increasing mode when the first button is pressed.

Please add your report here.

```c
#include "input_reading.h"

#define NO_OF_BUTTONS 3
#define BUTTON_IS_PRESSED GPIO_PIN_RESET
#define BUTTON_IS_RELEASED GPIO_PIN_SET

static GPIO_PinState buttonBuffer[NO_OF_BUTTONS];
static GPIO_PinState debounce1[NO_OF_BUTTONS];
static GPIO_PinState debounce2[NO_OF_BUTTONS];
static uint16_t buttonPin[NO_OF_BUTTONS] = {b1_Pin, b2_Pin,
    b3_Pin};

void button_reading_Init(void){
    for(int i = 0; i < NO_OF_BUTTONS; i++){
        debounce1[i] = BUTTON_IS_RELEASED;
        debounce2[i] = BUTTON_IS_RELEASED;
        buttonBuffer[i] = BUTTON_IS_RELEASED;
    }
}

void button_reading(void){
    for(int i = 0; i < NO_OF_BUTTONS; i++){
        debounce2[i] = debounce1[i];
        debounce1[i] = HAL_GPIO_ReadPin(GPIOB, buttonPin[i
    ]);
        if(debounce1[i] == debounce2[i]){
            buttonBuffer[i] = debounce1[i];
        }
    }
}

unsigned char is_button_pressed(uint8_t index){
    if(index >= NO_OF_BUTTONS) return 0;
    return (buttonBuffer[index] == BUTTON_IS_PRESSED);
}
```

Program 1.10: The code in the input$_reading.cfile$

5.7 Exercise 6: Adding code for displaying modes

Your tasks in this exercise are:

- To add code for display mode on seven-segment LEDs, and
- To add code for blinking LEDs depending on the mode that is selected.

Please add your report here.

```
void upmode(uint8_t mode){
    led_buffer[2] = 0;
    led_buffer[3] = mode;
}
    if(timer1_flag){
        timer1_flag = 0;
        scan7SEG();
        setTimer1(250);
    }
    if(is_button_pressed(0)){
        if(!button_flag[0]){
            button_flag[0] = 1;
            mode = mode + 1;
            if(mode > 4) mode = 1;
            turnOffTraffic();
            if(mode == 1){
                traffic_init();
            }
        }
    } else {
        button_flag[0] = 0;
    }

  void blink_leds_mode(void){
    static int blink_state = 0;

    if(timer2_flag){
        timer2_flag = 0;
        blink_state = !blink_state;
        setTimer2(100);

        switch(mode){
            case 2:
                HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8, !blink_state);
                HAL_GPIO_WritePin(GPIOA, GPIO_PIN_11, !blink_state);
                break;
            case 3:
                HAL_GPIO_WritePin(GPIOA, GPIO_PIN_9, !blink_state);
```

```
                HAL_GPIO_WritePin(GPIOA, GPIO_PIN_12, !blink_state);
                break;
            case 4:
                HAL_GPIO_WritePin(GPIOA, GPIO_PIN_10, !blink_state);
                HAL_GPIO_WritePin(GPIOA, GPIO_PIN_13, !blink_state);
                break;
            default:
                break;
        }
    }
}
```

Program 1.11: The code in the processing.c file

5.8 Exercise 7: Adding code for increasing time duration value for the red LEDs

Your tasks in this exercise are:

- to use the second button to increase the time duration value of the red LEDs
- to use the third button to set the value for the red LEDs.

Please add your report here.

```

        case 2:
            blink_leds_mode();
            if(is_button_pressed(1)){
                if(!button_flag[1]){
                    button_flag[1] = 1;
                    int new_red = (get_red() % 99) + 1;
                    set_traffic_time(new_red, get_yellow(), get_green());
                }
            } else {
                button_flag[1] = 0;
            }
            upclock(get_red());
            upmode(2);
            break;
```

Program 1.12: The code in the processing.c file

5.9 Exercise 8: Adding code for increasing time duration value for the amber LEDs

Your tasks in this exercise are:

- to use the second button to increase the time duration value of the amber LEDs
- to use the third button to set the value for the amber LEDs.

Please add your report here.

5.9.1 The code in the main.c file

```

        case 3:
            blink_leds_mode();
            if(is_button_pressed(1)){
                if(!button_flag[1]){
                    button_flag[1] = 1;
                    int new_yellow = (get_yellow() % 99) + 1;
                    set_traffic_time(get_red(), new_yellow, get_green());
                }
            } else {
                button_flag[1] = 0;
            }
            upclock(get_yellow());
            upmode(3);
            break;
```

Program 1.13: The code in the processing.c file

5.10 Exercise 9: Adding code for increasing time duration value for the green LEDs

Your tasks in this exercise are:

- to use the second button to increase the time duration value of the green LEDs
- to use the third button to set the value for the green LEDs.

Please add your report here.

```c

        case 4:
            blink_leds_mode();
            if(is_button_pressed(1)){
                if(!button_flag[1]){
                    button_flag[1] = 1;
                    int new_green = (get_green() % 99) + 1;
                    set_traffic_time(get_red(), get_yellow
    (), new_green);
                }
            } else {
                button_flag[1] = 0;
            }
            upclock(get_green());
            upmode(4);
            break;
```

Program 1.14: The code in the processing.c file

5.11 Exercise 10: To finish the project

Your tasks in this exercise are:

- To integrate all the previous tasks to one final project:
- To create a video to show all features in the specification:
- To add a report to describe your solution for each exercise.
- To submit your report and code on the BKeL

4. Tài nguyên

- **Code GitHub:** https://github.com/phucnguyen0705/VDK$_LAB3_2$312693
- **Video minh họa:** https://drive.google.com/drive/u/1/folders/101b59sIv6sW1oHPPEVV